Pulse Month 1 Week 1

Understand + Build a Raw TCP Server

How to use this document
Part 1 — C++ basics needed for systems programming
- Compilation and linking
- Header/source files
- Pointers vs. references
- Arrays vs. `std::vector`
- Strings: `std::string` vs `std::string_view`
- Error handling (no exceptions here)
- RAII

Part 2 — Linux concepts
- Process, user space, kernel space
- File descriptors, and stdin/stdout/stderr
- `/proc`

Part 3 — Networking concepts
- IP address, localhost, port
- TCP, and why it doesn't know HTTP
- TCP handshake

Part 4 — The syscalls, with real usage
- `socket()` — create an endpoint
- The address struct you fill in before `bind()`
- `bind()` — claim an address
- `listen()` — go passive, accept a queue of connections
- `accept()` — pull one connection off the queue
- `read()` / `write()` — move bytes over an established connection
- `close()` — release the FD
- `setsockopt()` — tweak socket behavior

Part 5 — Why there are two file descriptors
Part 6 — Questions I must be able to answer (with answers)
Part 7 — What you're building this week (recap)

How to use this document

This is reading material, not the assignment. It does **not** contain a working `server.cpp` — writing that is Week 1's actual exercise.

Every concept below gets three things: **what it is**, **an analogy** (blue box) so it clicks intuitively, and **a real example** (green box) — actual command output or actual code, not just prose. Read once end-to-end, then keep it as reference while you build.

Part 1 — C++ basics needed for systems programming

Compilation and linking

```
source.cpp  --[compiler]-->  object file (.o)  --[linker]-->  executable
```

The **compiler** turns one `.cpp` into machine code, on its own, without needing the *bodies* of functions from other files — just their declarations (from headers). The **linker** then stitches every object file together and resolves calls between them.

> **Analogy:** writing a book with co-authors. Each author (compiler) writes their chapter (object file) knowing only the *chapter titles* of the others (declarations in headers) — not their full text. An editor (linker) later assembles all chapters into one book and makes sure every cross-reference ("see Chapter 3") actually points at a chapter that exists.

Real example — why this distinction matters when debugging:

```
// math_utils.hpp
int square(int x);   // declaration only
```

```
// main.cpp
#include "math_utils.hpp"
int main() { return square(4); }
```

If `math_utils.cpp` (which defines `int square(int x) { return x * x; }`) is never compiled and linked in, you get:

```
undefined reference to `square(int)'
```

This is a **linker** error, not a compiler error — `main.cpp` compiled just fine because it only needed the declaration. Recognizing "linker error" vs "compiler error" immediately tells you whether the problem is a typo/syntax issue (compiler) or a missing/misconfigured source file in your build (linker) — this is exactly the kind of error you'll hit once your CMakeLists.txt has multiple `.cpp` files.

Header/source files

A header (`.hpp`) is the *contract*; the source (`.cpp`) is the *fulfillment*.

Real example:

```
// router.hpp — anyone who wants to use Router includes THIS
class Router {
public:
    std::string handle(const std::string& path);
};
```

```
// router.cpp — the actual logic, compiled once
#include "router.hpp"
std::string Router::handle(const std::string& path) {
    if (path == "/hello") return "Hello from Pulse!";
```

```
    return "404 Not Found";
}
```

```
// main.cpp — never sees router.cpp, only the header
#include "router.hpp"
int main() {
    Router r;
    std::cout << r.handle("/hello");
}
```

`main.cpp` compiles successfully even though it has never seen the actual `if` logic — it trusts the header's promise, and the linker connects the two `.o` files at the end.

Pointers vs. references

```
int x = 5;

int* p = &x;   // pointer: holds an ADDRESS. Can be null. Can be reassigned.
*p = 10;       // dereference — "go to that address and change what's there"
std::cout << x; // prints 10 — we changed x through p

int& r = x;    // reference: an ALIAS for x. Cannot be null. Cannot be rebound.
r = 20;        // no dereference syntax needed — acts exactly like x
std::cout << x; // prints 20
```

Analogy: a pointer is like writing someone's *street address* on a sticky note — you can erase it and write a different address, or write nothing (null). A reference is like giving someone a *nickname* — "Bob" always refers to the same actual person, it's just another name for something that already exists; you can't quietly repoint "Bob" at a different person.

Real example — why this matters for sockets: `read()`'s actual POSIX signature is `ssize_t read(int fd, void *buf, size_t count);` — that `void* buf` is a raw pointer because the kernel needs a memory address to write incoming bytes into; it has no concept of C++ references. Meanwhile, your own `Router::handle(const`

> `Request& req)` should take the parsed request *by reference* so you're not copying the whole request struct (headers, body, everything) every time you dispatch to a handler.

Arrays vs. `std::vector`

A raw array (`char buf[1024]`) doesn't know its own size at runtime — `sizeof(buf)` only works because the compiler still sees the declaration in that scope; pass it to a function as a pointer and that information is gone. `std::vector<char>` tracks its own size and can grow.

> **Real example:**
>
> ```
> void bad(char buf[]) {
>     std::cout << sizeof(buf); // prints 8 (pointer size), NOT 1024. Classic bug.
> }
>
> void good(const std::vector<char>& buf) {
>     std::cout << buf.size(); // actually correct, always
> }
> ```
>
> For Pulse specifically: you don't know an incoming HTTP request's length ahead of time (could be 40 bytes, could be 4000 with a big body) — that's a `std::vector<char>` you grow as more `read()` calls bring in more bytes, not a fixed-size array you hope is big enough.

Strings: `std::string` vs `std::string_view`

`std::string` **owns and allocates** its data. `std::string_view` is just `{pointer, length}` pointing at data owned by *someone else* — no allocation.

> **Real example:**
>
> ```
> std::string raw = "GET /hello HTTP/1.1\r\n";
>
> // Owning copy — allocates new memory:
> ```

```
std::string method_owned = raw.substr(0, 3);

// Non-owning view — zero allocation, just {pointer into raw, length=3}:
std::string_view method_view(raw.data(), 3);
```

The danger:

```
std::string_view danger() {
    std::string local = "temporary";
    return std::string_view(local);   // local is destroyed when function returns!
}
// caller now holds a view pointing at freed memory — undefined behavior
```

When you parse an HTTP request out of your read buffer, slicing out the method/path as `string_view`s into that same buffer is efficient — but only as long as the buffer itself is still alive. This is a real bug category, not a theoretical one.

Error handling (no exceptions here)

POSIX calls return `-1` on failure and set `errno`. You check the return value:

Real example — actual pattern you'll write in Week 1:

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
if (fd == -1) {
    perror("socket");   // e.g. prints: socket: Address family not supported
    return 1;
}
```

Try this yourself once you're in Ubuntu: run a program that calls `bind()` on a port your server is *already* bound to in another terminal. `perror("bind")` will print `Address already in use` — that's `errno` being translated to English for you, straight from the kernel.

RAII

Tie a resource's lifetime to an object's lifetime so it's released automatically — even on early `return` or an exception.

> **Analogy:** a hotel key card that self-deactivates the moment you walk out of the room, instead of you having to remember to hand it back at the front desk every single time (including if you leave in a hurry through the fire exit).

Real example — the bug RAII prevents:

```
void handle_client(int client_fd) {
    char buf[1024];
    ssize_t n = read(client_fd, buf, sizeof(buf));
    if (n < 0) {
        perror("read");
        return;              // BUG: client_fd never closed. Leaked.
    }
    write(client_fd, "hi", 2);
    close(client_fd);        // only reached on the happy path
}
```

A tiny RAII wrapper fixes every early-return path at once:

```
class Fd {
public:
    explicit Fd(int fd) : fd_(fd) {}
    ~Fd() { if (fd_ != -1) close(fd_); }   // runs on EVERY exit path
    int get() const { return fd_; }
private:
    int fd_;
};
```

You don't have to build this in Week 1 — just notice, while you write your accept loop, every place you `close()` and ask "what if I returned/broke out before reaching this line?"

Part 2 — Linux concepts

Process, user space, kernel space

> **Analogy:** user space is the dining area of a restaurant — customers (your programs) can request things, but can't walk into the kitchen and grab a knife themselves. Kernel space is the kitchen — it has the actual dangerous, powerful equipment (hardware access), and only kitchen staff (the kernel) are allowed to operate it. A system call is you telling the waiter "I'd like the steak" — a controlled, well-defined request across that boundary.

> **Real example:** once your server is running, open a second terminal and run:
>
> ```
> ps aux | grep pulse
> ```
>
> You'll see a real line like:
>
> ```
> you   4821  0.0  0.1  8912  1780 pts/1  S+  10:02   0:00  ./pulse 8080
> ```
>
> `4821` is the PID — a process, currently sitting in user space, that has asked the kernel (via `accept()`) to block until a client connects. It is not touching the network card directly; it's waiting on the kernel to do that and wake it up.

File descriptors, and stdin/stdout/stderr

> **Analogy:** a coat check ticket. The number on your ticket ("14") means nothing on its own — it's only meaningful to *that specific coat check counter* (your process), which has the real mapping of "ticket 14 → the blue jacket on hook 14." Someone else's ticket 14 at a different counter (a different process) refers to a completely different coat.

> **Real example:** write a one-line C program that does nothing but sleep, run it, then inspect it:
>
> ```
> sleep 300 &
> ls -l /proc/$!/fd
> ```
>
> Real output looks like:
>
> ```
> lrwx------ 1 you you 64 Sep 13 10:02 0 -> /dev/pts/1
> lrwx------ 1 you you 64 Sep 13 10:02 1 -> /dev/pts/1
> lrwx------ 1 you you 64 Sep 13 10:02 2 -> /dev/pts/1
> ```
>
> FDs 0, 1, 2 — stdin/stdout/stderr — already exist, all pointing at your terminal. This is *why*, once your Pulse server calls `socket()`, the FD it gets back will typically be `3` — the first three slots were already taken before your code ran a single line.

`/proc`

Not real files on disk — the kernel exposing live internal state *as if* it were a filesystem, generated on the fly when you read it.

> **Real example, directly relevant to Week 1:** once Pulse is running and listening, find it and inspect it live:
>
> ```
> ss -ltnp
> ```
>
> ```
> State   Recv-Q  Send-Q   Local Address:Port   Peer Address:Port  Process
> LISTEN  0       128            0.0.0.0:8080        0.0.0.0:*      users:
> (("pulse",pid=4821,fd=3))
> ```
>
> That last column is the kernel telling you, live: PID 4821 owns this listening socket, and it's sitting at FD 3 in that process — confirming exactly what the previous example predicted.

Part 3 — Networking concepts

IP address, localhost, port

> **Analogy:** an IP address is an apartment building's street address. A port is the specific apartment number. "127.0.0.1" is special — it's like mailing something to yourself: it never leaves the building, it just gets rerouted straight back inside.

Real example:

```
curl -v http://127.0.0.1:8080/hello
```

```
*   Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080
> GET /hello HTTP/1.1
> Host: 127.0.0.1:8080
```

`curl` had to know *both* the address (which machine — trivial here, it's your own) and the port (which process on that machine — one of potentially dozens listening on your machine right now). Try `curl http://127.0.0.1:9999/hello` with nothing listening on 9999 — you'll get `Connection refused`, because the kernel checked and no process owns that apartment number.

TCP, and why it doesn't know HTTP

> **Analogy:** TCP is like a phone line guaranteed to deliver your words in the order you said them, with no garbling — but the phone company has no idea if you're speaking English, ordering pizza, or reciting poetry. It just guarantees the *sound* arrives intact and in order. What the sounds *mean* is entirely between you and the person on the other end.

> **Real example, proving it:** once your Week 1 server is running, connect with raw `nc` instead of a browser and type garbage:
>
> ```
> nc localhost 8080
> ```
>
> ```
> asdkfj not http at all lol
> ```
>
> Your TCP connection establishes completely fine — the kernel handshake succeeds, bytes flow, `read()` returns them successfully. It's only *your own code*, if you'd written an HTTP parser, that would look at those bytes and say "this isn't a valid HTTP request." TCP itself never objects — it delivered exactly the bytes it was asked to, faithfully. That's the proof that "TCP is a dumb pipe, HTTP is a meaning you and the client agree to impose on top of it."

TCP handshake

```
Client                    Server
  | ------ SYN ----------->  |     "I'd like to connect"
  | <---- SYN-ACK ---------  |     "OK, acknowledged, same to you"
  | ------ ACK ------------> |     "Acknowledged, we're connected"
```

> **Real example — watch it actually happen:** in Ubuntu, run (may need `sudo apt install tcpdump`):
>
> ```
> sudo tcpdump -i lo port 8080 -n
> ```
>
> Then in another terminal, `curl http://localhost:8080/hello`. You'll see real packet lines like:
>
> ```
> IP 127.0.0.1.51322 > 127.0.0.1.8080: Flags [S], seq 123456789
> IP 127.0.0.1.8080 > 127.0.0.1.51322: Flags [S.], seq 987654321, ack 123456790
> IP 127.0.0.1.51322 > 127.0.0.1.8080: Flags [.], ack 987654322
> ```

> `[S]` = SYN, `[S.]` = SYN-ACK, `[.]` = ACK. That's the three-step handshake from the diagram above, captured as it actually happened on your machine, before `accept()` even returned.

Part 4 — The syscalls, with real usage

For each syscall: the full signature, what every single parameter means (with the actual values you'll pass), what it returns and how to tell success from failure, and a use case explaining *why* that parameter exists at all. Look up the man pages yourself too (`man 2 <name>` in Ubuntu) — that's expected, not cheating, this just gets you oriented first.

`socket()` — create an endpoint

```
int socket(int domain, int type, int protocol);

int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
```

Parameter	What it means	What you'll actually pass	Why it exists / use case
`domain`	Which address family — i.e. what *kind* of address this socket will use	`AF_INET` (IPv4 addresses like `127.0.0.1`)	`AF_INET6` exists for IPv6; `AF_UNIX` exists for local-machine-only sockets identified by a filesystem path instead of an IP+port (used for things like Docker's `/var/run/docker.sock`). One `socket()` call has to support all of these, so it needs to be told up front which addressing scheme applies.

Parameter	What it means	What you'll actually pass	Why it exists / use case
`type`	What communication style — reliable stream vs. unreliable datagrams	`SOCK_STREAM`	`SOCK_STREAM` = TCP-like: ordered, reliable, connection-based byte stream — what HTTP needs. `SOCK_DGRAM` = UDP-like: fire-and-forget individual packets, no ordering/delivery guarantee, no connection setup — wrong choice for HTTP, but it's why this parameter exists: the same `socket()` call is the entry point for building either kind of network primitive.
`protocol`	Which specific protocol under that domain+type combination	`0`	`0` means "pick the only sensible default for this domain+type" — for `(AF_INET, SOCK_STREAM)` that default is TCP, so `0` and the explicit constant `IPPROTO_TCP` behave identically here. You'd only pass something explicit if a domain+type pair were ambiguous between multiple protocols (rare in practice for what you're building).

Return value: the new file descriptor (an integer ≥ 0) on success, or `-1` on failure (check `errno`, e.g. `EPROTONOSUPPORT`, `EMFILE` if you've hit your process's open-file limit).

Use case recap: this call *only* allocates the socket and hands you an FD — nothing is bound, nothing is listening, no address is attached. It's the equivalent of getting an unassigned phone handset.

The address struct you fill in before `bind()`

Before `bind()`, you build a `struct sockaddr_in` describing the local address you want to claim:

```
struct sockaddr_in addr{};
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = INADDR_ANY;   // "any local interface"
addr.sin_port = htons(8080);         // htons: host byte order -> network byte order
```

Field	What it means	What you'll actually pass	Why it exists / use case
`sin_family`	Must match the `domain` you gave `socket()`	`AF_INET`	The kernel double-checks this matches, since the rest of the struct's layout depends on which address family it is.
`sin_addr.s_addr`	Which **local network interface** to bind to	`INADDR_ANY`	`INADDR_ANY` means "accept connections arriving on *any* of this machine's interfaces" (localhost, its LAN IP, etc. — all of them). You'd bind

Field	What it means	What you'll actually pass	Why it exists / use case
			to one *specific* IP instead if a machine has multiple network interfaces (e.g. a VPN and a LAN) and you deliberately want your server reachable through only one of them.
`sin_port`	Which port number to claim	`htons(8080)`	This is *why* the port matters at all — see Part 3. It must go through `htons()` (host-to-network byte order) because your CPU (x86, little-endian) and the network protocol (big-endian, "network byte order") disagree on how multi-byte numbers are laid out in memory; skip this and the kernel reads your `8080` as a completely different number.

> **Real example of what happens if you skip byte-order conversion:** forget `htons()` and `8080` in memory might get interpreted as port `36895` instead. Try commenting it out and running `ss -ltnp` afterward — you'll likely see a wrong-looking port number bound, proving the byte-order issue is real, not theoretical.

Why `struct sockaddr*` and not `struct sockaddr_in*` in the function signatures below? `bind()`, `accept()`, and friends were designed once to work for *every* address family (`AF_INET`, `AF_INET6`, `AF_UNIX`, …), each of which needs a differently-shaped struct. `struct sockaddr` is a generic "at least this big, first field is always the family" placeholder type — you build the real, family-specific struct (`sockaddr_in` for IPv4) and cast a pointer to it down to the generic type. It's C's pre-object-oriented way of doing polymorphism: the function reads `sin_family` first to know how to reinterpret the rest of the bytes.

`bind()` — claim an address

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

bind(listen_fd, (struct sockaddr*)&addr, sizeof(addr));
```

Parameter	What it means	What you'll actually pass	Why it exists / use case
`sockfd`	The socket FD to attach the address to	`listen_fd` from `socket()`	Ties this specific socket to a specific local address — nothing else changes about the FD.
`addr`	Pointer to the address struct you built above, cast to the generic type	`(struct sockaddr*)&addr`	See the polymorphism explanation above — the cast is required because that's the type the

Parameter	What it means	What you'll actually pass	Why it exists / use case
			function signature expects, even though you built a concrete `sockaddr_in`.
`addrlen`	The actual size, in bytes, of the concrete struct you passed	`sizeof(addr)`	Since `sockaddr` is generic, the kernel needs to be told separately how many bytes it's actually safe to read from that pointer — `sockaddr_in` and `sockaddr_in6` aren't the same size.

Return value: `0` on success, `-1` on failure. Common `errno` values: `EADDRINUSE` (something already bound this port — see `SO_REUSEADDR` below), `EACCES` (trying to bind a port below 1024 without root privileges — this is why production servers often run on 8080 during development and get proxied from 80 in production).

`listen()` — go passive, accept a queue of connections

```
int listen(int sockfd, int backlog);

listen(listen_fd, 128);
```

Parameter	What it means	What you'll actually pass	Why it exists / use case
`sockfd`	The bound socket to make passive	`listen_fd`	Before this call, this FD can't receive incoming connections at all — connecting to it gives `Connection refused`.
`backlog`	Max number of connections that have **finished the TCP handshake** but that your code hasn't `accept()`ed yet	`128` (a reasonable default)	If clients connect faster than your program calls `accept()`, the kernel queues the completed handshakes up to this limit. Once full, further connection attempts get refused/dropped depending on OS config. A single-threaded, one-client-at-a-time Week 1 server with a small backlog is fine for testing; a real production server under heavy simultaneous load would tune this much higher.

Return value: `0` on success, `-1` on failure.

`accept()` — pull one connection off the queue

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);

struct sockaddr_in client_addr{};
socklen_t len = sizeof(client_addr);
int client_fd = accept(listen_fd, (struct sockaddr*)&client_addr, &len);
```

Parameter	What it means	What you'll actually pass	Why it exists / use case
`sockfd`	The listening socket to pull a connection from	`listen_fd`	Stays open and reusable for the next `accept()` call — see Part 5.
`addr`	**Output** parameter — the kernel fills this in with the *client's* address	`(struct sockaddr*)&client_addr`	Lets you find out who connected (their IP and ephemeral port) — useful for logging, rate-limiting, access control. Pass `NULL` here (and `NULL` for `addrlen`) if you genuinely don't care who connected.
`addrlen`	An **in/out** pointer: before the call, you set	`&len`, pre-set to `sizeof(client_addr)`	This in/out trick exists because, again, different

Parameter	What it means	What you'll actually pass	Why it exists / use case
	`*addrlen` to the size of the buffer you're providing; after the call, the kernel overwrites it with the actual size of the address it wrote		address families produce differently-sized addresses — the kernel needs to know your buffer's capacity going in, and tell you how much it actually used coming out.

Return value: a **new** FD representing this one connection, or `-1` on failure. This call **blocks** — your program's execution pauses here, doing nothing, until a client actually connects. That blocking behavior is exactly what "blocking I/O" means, and it's why a single-threaded Week 1 server can only usefully talk to one client at a time (handling many concurrently, without blocking, is deliberately out of scope until later months — see the epoll/async items in the Month 1 scope list).

> **Real example:** print `inet_ntoa(client_addr.sin_addr)` and `ntohs(client_addr.sin_port)` after this call. Connect with two separate `nc` sessions and you'll see two different ephemeral client port numbers — concrete proof each connection is a distinct, independently-tracked thing, even though both came from `127.0.0.1`.

`read()` / `write()` — move bytes over an established connection

```
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);

char buf[4096];
ssize_t n = read(client_fd, buf, sizeof(buf));
```

Parameter	What it means	What you'll actually pass	Why it exists / use case
`fd`	Which connection to read from / write to	`client_fd` (never `listen_fd` — see Part 5)	You never read/write on the listening socket; it has no data of its own, only new connections.
`buf`	Pointer to memory: for `read`, where the kernel should copy incoming bytes *to*; for `write`, where the kernel should copy outgoing bytes *from*	`buf` (a `char[]` or `std::vector<char>::data()`)	Raw `void*` because the kernel doesn't know or care about your C++ types — it just copies bytes.
`count`	The **maximum** number of bytes to transfer this call	`sizeof(buf)`	For `read`, this is your buffer's capacity, not a promise of how many bytes you'll get — see below. For `write`, it's how much of your data you're offering to send this call — also not a guarantee it

Parameter	What it means	What you'll actually pass	Why it exists / use case
			all goes at once.

Return value (the part that trips people up): an integer `n`. - `n > 0`: for `read`, you actually received `n` bytes — which can be **less than** `count`, even mid-message (TCP has no concept of "message boundaries" — see Part 3). For `write`, you actually sent `n` bytes, which can also be less than `count` for large buffers — a real HTTP response of any size should be written in a loop until all bytes are confirmed sent, not assumed to go out in one call. - `n == 0` (read only): the peer closed their end of the connection — not an error, a legitimate "no more data is ever coming" signal. - `n == -1`: an error occurred; check `errno` (e.g. `EINTR` means the call was interrupted by a signal and can usually just be retried).

> **Real example — proving read() doesn't respect message boundaries:** this is Week 3's big experiment, but you can preview it now. Write a tiny Python client:
>
> ```
> import socket, time
> s = socket.create_connection(("localhost", 8080))
> s.send(b"GET /hel")
> time.sleep(1)
> s.send(b"lo HTTP/1.1\r\nHost: x\r\n\r\n")
> ```
>
> If your Week 1 server just does one `read()` and prints what it got, it will print `GET /hel` — a truncated, syntactically incomplete line — proving a single `read()` call handed you exactly what had arrived so far, nothing more, nothing "smart."

`close()` — release the FD

```
int close(int fd);

close(client_fd);
```

Parameter	What it means	What you'll actually pass	Why it exists / use case
`fd`	The FD to release back to the kernel	`client_fd` (and eventually `listen_fd` on shutdown)	For a socket, this begins connection teardown (a FIN gets sent to the peer) and frees the kernel-side bookkeeping for that FD.

Return value: `0` on success, `-1` on failure (rarely checked for sockets, more relevant for regular files where a delayed write error can surface here) — but *always call it* on every code path. Skip it on an error path and that FD is leaked for the life of the process; leak enough of them and you'll hit `EMFILE: Too many open files`.

`setsockopt()` — tweak socket behavior

Why does this function need to exist at all? `socket()`, `bind()`, `listen()`, `accept()` each answer "what lifecycle stage is this socket in" — unallocated → bound → passively listening → connected. But a socket also has dozens of independent *behavioral knobs* that have nothing to do with lifecycle stage: buffer sizes, whether to allow rebinding an address stuck in `TIME_WAIT`, keepalive probes, close/linger behavior, and more — most of which almost no one ever needs to touch (defaults are fine). Giving every single one of those knobs its own dedicated function (`enable_reuseaddr()`, `set_recv_buffer_size()`, ...) would mean adding a brand new syscall to the kernel every time someone invented one more optional behavior — that doesn't scale. So instead there's **one generic "configure a socket setting" syscall**, and every optional behavior — old or newly invented — is just another named constant (`optname`) plugged into the same function, never requiring a new syscall.

> Think of a router's admin settings page. Plugging it in and connecting to your ISP (`socket` / `bind` / `listen`) is a small, fixed lifecycle every router goes through. But the settings page has a huge, ever-growing list of independent optional toggles (QoS, guest network, port forwarding...) that don't fit that sequence at all. Rather than a dedicated physical button for every possible setting, there's one generic form: pick

> which setting (`optname`), enter a value (`optval` / `optlen`). `setsockopt()` is that settings form for a socket — you'll only call it for the handful of behaviors where the default isn't what you want.

```
int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);

int opt = 1;
setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

Once you know *that* it's a generic settings-setter, the parameters make sense as "which setting, and what value": `optname` picks the setting; `optval` + `optlen` describe the value to set it to, using the only way a single C function can accept values of wildly different types and sizes — a raw memory address (`optval` , "here's a box containing the value") plus a byte count (`optlen` , "and the box is this many bytes"). Compare two calls: `SO_REUSEADDR` and `SO_RCVBUF` both pass `&some_int, sizeof(int)` — identical shape, totally different meaning, because the meaning comes from `optname` , not from the value's shape. `SO_LINGER` instead passes the address of a whole `struct linger` with `optlen = sizeof(struct linger)` — proof that `optlen` genuinely has to vary per option, since the "box" isn't always 4 bytes.

Parameter	What it means	What you'll actually pass	Why it exists / use case
`sockfd`	The socket to configure	`listen_fd`	Must be set **before** `bind()` — it affects how `bind()` behaves.
`level`	Which layer of the network stack this option belongs to	`SOL_SOCKET`	`SOL_SOCKET` = generic socket-level options (like `SO_REUSEADDR`); other levels exist for protocol-specific options (e.g. `IPPROTO_TCP` for TCP-specific

Parameter	What it means	What you'll actually pass	Why it exists / use case
			tuning like `TCP_NODELAY`, which you don't need yet).
`optname`	Which specific option to set	`SO_REUSEADDR`	Tells the kernel "allow re-binding this address even if it's technically still lingering in `TIME_WAIT` from a previous process."
`optval`	Pointer to the value to set the option to	`&opt`, where `opt = 1`	Options are set via a generic `void*` for the same reason addresses are: one function signature has to support options of many different types/sizes. Here it's a boolean-as-int: `1` = enable.
`optlen`	Size in bytes of the value pointed to by `optval`	`sizeof(opt)`	Same in/out-sizing reasoning as everywhere else in this API — the kernel needs to know how many bytes it's safe to read from `optval`.

Why isn't naming the option (`optname`) enough — why does `optval` have to exist too? Because `setsockopt` is one function shared by *every* option, and not all options are on/off switches. `SO_REUSEADDR` happens to be boolean, but `SO_RCVBUF` wants an actual byte count, and `SO_LINGER` wants a whole struct — the function has no idea from `optname` alone what *kind* of value a given option expects, it just blindly copies `optlen` bytes from `optval`. If merely naming the option meant "turn it on," that would only make sense for boolean options and would break entirely for `SO_RCVBUF` (there's no "on" for a buffer size). So the API needs one uniform rule that works for all of them: *the value at `optval` becomes the new setting* — no special-casing some options to "presence = enable."

That also gives boolean options like `SO_REUSEADDR` a way to be switched back off later, which "just naming it" could never do (you can't call a function to signal "I am now *not* calling it"):

```
int on = 1;  setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &on, sizeof(on));  // enable
int off = 0; setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &off, sizeof(off)); // disable
```

Both directions go through the exact same call shape — only possible because `optval` carries a real value instead of the call's mere presence meaning something.

> **Real example of the problem this solves:** stop your server with Ctrl+C, then immediately restart it. Without this call, you'll often get:
>
> ```
> bind: Address already in use
> ```
>
> even though the old process is gone — the kernel holds the port in `TIME_WAIT` for a short period after close, in case delayed packets from the old connection are still in flight. `SO_REUSEADDR` tells the kernel "let me rebind anyway." Try removing this call later and reproducing the error on purpose — it's a good way to actually believe `TIME_WAIT` is real instead of just reading about it.

Common misread — worth being precise about: this is *not* "let two different processes both actively use the same port at once." Only one live socket is ever actually receiving traffic on a given port at a time. `SO_REUSEADDR` only relieves the kernel's caution about a port whose *old, already-dead* socket hasn't fully aged out of `TIME_WAIT` yet — it's about

faster turnover of a now-empty slot, not concurrent occupancy. (A different option, `SO_REUSEPORT`, is what actually lets multiple *live* sockets bind the same port simultaneously, for load-balancing across processes/threads — out of scope for Month 1.)

Part 5 — Why there are two file descriptors

```
Listening socket (e.g. FD 3)        Client socket (e.g. FD 4)
   - bound to (0.0.0.0, 8080)          - returned by accept()
   - stays open, keeps listening       - represents ONE connection
   - never read()/write()'d on         - this is what you read()/write() on
                                        - closed when that conversation ends
```

> **Analogy:** the listening socket is the receptionist's front desk phone number — it never actually holds a conversation, it just keeps ringing for new callers. Each time a call comes in, the receptionist transfers it to a fresh extension (a new FD) where the actual conversation happens, freeing the front desk number to receive the next call immediately.

> **Real example:** open two `nc localhost 8080` sessions from two terminals against a server that logs the FD number it gets from `accept()`. You'll see two different numbers (e.g. 4, then 5) while `ss -ltnp` still shows the original listening FD (3) unchanged and still in `LISTEN` state the whole time.

Part 6 — Questions I must be able to answer (with answers)

Cover the answer, try it yourself first.

Why did `socket()` return an integer? Because Linux represents every kernel-managed handle — files, pipes, sockets — as a small integer index into your process's private FD table. Uniform, cheap, and it's the same mechanism `open()` uses for regular files.

What does that integer represent? An index into *your process's* table of open descriptors, maintained by the kernel — not the socket itself, just your handle to it.

Why does the server need `bind()`? A freshly created socket has no address. Without `bind()`, the kernel has no way to know that packets arriving on port 8080 should be routed to *this* socket.

What does `listen()` actually change? It switches the socket from "usable for outgoing connections only" to "passive, willing to accept incoming ones," and allocates the backlog queue for handshake-completed connections awaiting `accept()`.

Why does `accept()` return another FD? Because the listening socket and one client's conversation are different-lived things — the former persists across many clients, the latter is tied to a single connection that needs to be read, written, and closed independently.

Why are there two FDs? So the server can keep listening for new clients while still holding an open conversation with an existing one — see Part 5's phone receptionist analogy.

What is the difference between the listening socket and the client socket? Listening socket: bound to a local address, only ever used to accept connections, never read/written. Client socket: represents one specific established connection, is what you actually exchange bytes through.

What happens when the client disconnects? The client's OS sends a TCP FIN. Your next `read()` on that FD returns `0` — not an error, not blocking — signaling "no more data will ever arrive here." You then `close()` your side.

Part 7 — What you're building this week (recap)

```
client --"hello"--> server --"hello"--> client
```

Steps: `socket()` → `bind()` → `listen()` → `accept()` one client → `read()` → print → `write()` a response → `close()`.

Test with `nc localhost 8080`. Inspect with `ss -ltnp` and `ls -l /proc/<PID>/fd`. Don't move to Week 2 until you can explain Part 6 out loud, unaided.